

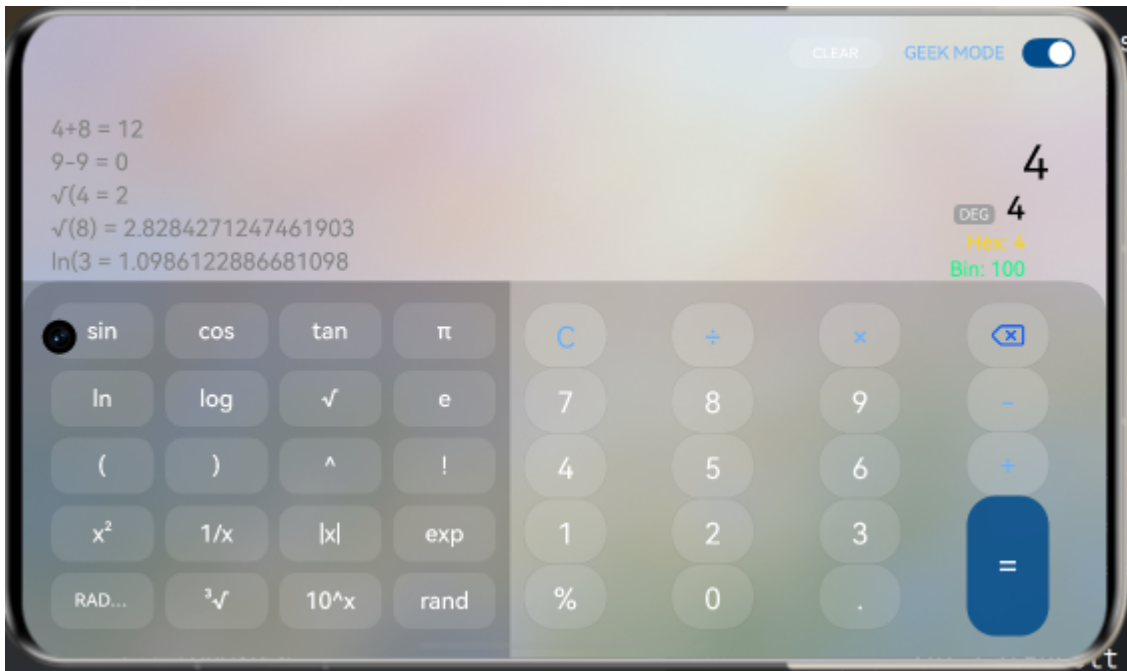
实验课程	中国海洋大学26夏《移动软件开发》
实验名称	实验5：鸿蒙开发入门及计算器开发
GitHub仓库地址	https://github.com/Irina-yang/OUC-Mobile-Software-Dev
博客地址	https://blog.csdn.net/xiaoyang_0621/article/details/164507186?sharetype=blogdetail&shareId=164507186&sharerefer=PC&sharesource=xiaoyang_0621&spm=1011.2480.3001.8118

一、实验内容

本实验旨在基于 HarmonyOS 和 ArkTS 框架，综合运用声明式 UI 开发、状态管理及组件逻辑处理等核心技术，独立设计并开发一款具备高可用性的“多功能智能计算器”。

在圆满完成基础四则运算等核心实验要求的基础之上，为进一步提升应用的工程完整度与用户交互体验，本人在本次实验中自主设计并实现了以下扩展功能（发挥部分）：

- 1. **精细化响应式 UI 架构与安全区域适配**：利用媒体查询（`mediaquery`）实现横竖屏自适应布局。竖屏状态呈现大尺寸基础数字键盘，横屏状态自动平滑扩展为 `5x4` 科学计算矩阵。此外，通过精准的 `Padding` 边距控制，完美适配物理设备的安全区域（有效规避刘海与前置摄像头导致的 UI 截断），并融合了毛玻璃（`BlurStyle`）等现代高质感视觉设计。
- 2. **强健的科学计算器解析引擎**：重构了底层计算逻辑，自主实现了一套支持“算式步骤可视化”的智能解析引擎。支持多维度的科学运算（三角函数、对数、根号、阶乘、指数等），具备自动补全缺失括号、实时兼容大数字科学计数法的功能，并加入了 RAD/DEG（弧度/角度）状态实时切换指示，极大增强了计算器的专业逻辑处理能力。
- 3. **极客模式 (Geek Mode) 数据转换**：面向计算机及网络安全专业背景，首创了“极客模式”。通过顶部 `Toggle` 开关，系统利用 `@watch` 实时监听当前的十进制计算结果，底层调用位运算（`>>> 0`）动态计算，并在界面上实时同步显示该数值的无符号十六进制（Hex）和二进制（Bin）格式。
- 4. **动态历史记录栈与一键清理 (CLEAR)**：在界面上方构建了专属滑动区域（`scroll`）用于展示历史计算过程，配合 `Scroller` 控制器实现了新记录产生时自动平滑滚动到底部的交互。同时在顶部导航栏引入动态条件渲染，当存在历史数据时自动展现“CLEAR”按键，实现数据的一键清空。
- 5. **基于 PersistentStorage 的状态持久化 (重启保留)**：突破了应用单次生命周期的内存限制，深入运用 ArkTS 的 `PersistentStorage` 机制与 `@StorageLink` 装饰器，实现历史记录数据栈与设备本地磁盘的双向绑定。即使强制杀掉应用进程或重启计算器，之前的计算历史记录依然完整保留。



二、问题总结与体会

1. 实验过程中遇到的问题及解决方案

- **问题一：横屏响应式布局导致底层键盘被挤压截断**
 - **描述：**在横屏状态下，由于设备垂直高度骤减，上方显示区域与历史记录区域占用了过多高度，导致底部 `Grid` 键盘的 0 键和等号键超出屏幕可视范围。此外，靠边缘的文字与按键容易与手机刘海/摄像头重叠。
 - **解决：**通过三元运算符 `this.isLandscape ? A : B` 极限压缩横屏状态下的显示区高度（如从 280vp 压缩至 140vp），限制输入框的 `maxLines`，并对外层 `column` 设置了非对称的 `padding`（如 `right: 40`）以规避物理挖孔。将底部的 `FlexAlign.Bottom` 语法修正为 ArkTS 标准的 `FlexAlign.End`，使布局重新对齐。
- **问题二：ArkTS 严格类型检查导致复杂对象数组报错**
 - **描述：**在抽离货币和单位换算数据时，直接使用 `[ {name: '人民币', unit: 'CNY', rate: 1.0} ]` 触发了 `arkts-no-untyped-obj-literals` 编译报错。
 - **解决：**深刻理解了 ArkTS 相比标准 TS 更严格的类型校验机制。通过显式声明 `class UnitItem` 数据结构，并改用 `new UnitItem(...)` 进行数组实例化，彻底消除了无推断类型的编译红线。对于正则替换中的未使用参数，采用了 `_match` 的下划线前缀规范进行规避。
- **问题三：科学计算解析引擎陷入死循环（AppFreeze）与浮点数精度崩溃**
 - **描述：**在实现“算式步骤可视化”时，由于 ArkTS 禁用 `eval()`，手动编写的 `while(str.includes('('))` 递归正则解析器在遇到不完整算式（如单独输入 `sin()`）时无法匹配替换，导致 `while` 陷入无限死循环卡死主线程。同时，`Math.sin(Math.PI)` 因 JS 浮点误差计算出 `-1.477535e+1` 的离谱结果。
 - **解决：**在所有的 `while` 解析循环中植入了安全锁（`if (str === prevStr) break;`），一旦发现字符串不再变化强制跳出。针对浮点误差，封装了 `formatFloat` 极小值归零函数（拦截 `< 1e-10` 的数值）；针对大数报错，升级了底层的正则表达式，使其能够正确识别和切分带有 `e` 的科学计数法。

2. 收获与体会

本次实验完整经历了从需求拆解、UI 还原到核心算法重构的大前端开发全生命周期。最大的收获在于突破了声明式 UI 的表层应用，深入理解了状态驱动视图的核心理念（如 `@State` 监听数组重绘，`@StorageLink` 同步硬盘数据）。在重构智能解析引擎的过程中，深刻体会到了“防御性编程”的重要性——不仅要让程序在正确输入下跑通，更要考虑残缺输入、极端大数、浮点误差等边界条件对系统稳定性的致命影响。

此外，在开发过程中将 Google AI Studio 辅助开发（Vibe Coding）融入 workflow，通过精准描述报错堆栈、提供 UI 对标图以及拆解逻辑步骤，大幅提升了 Debug 和底层正则重构的效率，这种人机协同的开发模式为未来的复杂项目工程化提供了极大的助力。